CMSC701, 2024, pp. 1–7

doi:
Paper

PAPER

FFIndex: Compressed Indexing for Parallel FAST-Q Parsing

Siddhant Bharti,[1] Prajwal Singhania,[1] Rakrish Dhakal[1] and Rob Patro[1]

[1]Department of Computer Science, University of Maryland, College Park, 20740, Maryland, USA

Abstract

Our project's aim is to address the challenge of efficiently parsing compressed `FASTQ` files. This is critical for genomic data processing pipelines but is hindered by the sequential nature of current decompression and parsing methods. The goal is to develop a tool and accompanying library that facilitates parallel processing by creating and utilizing an auxiliary index over compressed `FASTQ` files. We demonstrate how just keeping the 32KB uncompressed data before a series of access-points allows us to deterministically decompress the compressed `FASTQ` file in parallel. This leads to speedups in decompression. In the results section, we show that with multiple reader-threads, the processing time can be significantly reduced. We compare the speedups in our method to different baselines. The code for this project can be found in our GitHub repository: `https://github.com/siddhant-bharti/CMSC701-Project`

Introduction

Many contemporary genomics algorithms and tools are embarrassingly parallel, and modern processors have many cores that can support dozens to hundreds of simultaneous threads of execution. Ingesting and parsing the raw read records is a significant bottleneck for these algorithms and tools. The reasons for this are manyfold: *1*) Variable record lengths in `FASTQ` files can impede scaling because identifying these record boundaries needs to be done sequentially; *2*) Synchronization and locking while reading a single `FASTQ` file by many threads can significantly affect performance; *3*) `FASTQ` files are often stored in gzipped compressed formats which makes reading these files using many threads hard since decompression is a sequential process. . [9] provides a good analysis over these bottlenecks for the
FASTQ files.

With continued advancements in sequencing, the amount of data being generated, and the corresponding sizes of the `FASTQ` files being produced is increasing. For example, Kerbiriou et al. [6] demonstrate results in their work on attempting to parallelize decompression on `FASTQ` files around 37GBs in size. Therefore these files are stored and shared in compressed format. Gzip has shown to achieve good size reduction for `FASTQ` files [6] while still offering reasonably fast decompression. However, Gzip's DEFLATE [1] (LZ77 and Huffman coding) algorithm is sequential. This makes random access in compressed `FASTQ` files hard.

Block compression in this format is possible, whereupon files are first broken into chunks, and then these chunks are compressed separately. This can allow parallel decompression of the compressed file but it raises two additional issues. First, such an approach can negatively affect the compression quality obtained by the compression algorithm. Second, even if blocks are compressed independently, there is no guarantee that they are "record aligned" (i.e. that chunks start and end on `FASTQ` record boundaries, or that they blocks in paried-end files are synchronized and each contain the same number of records).

This bottleneck is further underscored by the fact that reading compressed data is a slow process. *gunzip*, the decompressor component of *gzip*, can read compressed data at around 30-50 MB/sec which is order of magnitude less than read throughput of current SATA/NVMe solid-state drives [6].

The compression and decompression algorithms used in Gzip are inherently serial. As such, existing `FASTQ` parsers can only begin reading starting from the beginning of each compressed `FASTQ` file. However, if we know the relevant context of the compressor (32KB of uncompressed data before the access point of interest), then we can then start decompressing only from that point onward. This is the idea behind our method. We create certain "access points" and store the preceding 32KB of uncompressed data to be able to decompress from that point forward. Using this idea, we can have multiple threads read the compressed `FASTQ` file, where each thread starts processing from an access point and read some number of records.

We have created `FFIndex`, the tool that creates this index over compressed `FASTQ` files, as well as `FFParser`, a corresponding (`C++`) library that uses this index to be able to read the compressed `FASTQ` file in parallel. The idea is to incur a one-time cost of building the index, which then significantly accelerates parsing compressed `FASTQ` files in the future. This index can be shared alongside the `FASTQ` files in repositories like the European Nucleotide Archive [16], which amortizes the cost of the index creation. Moreover, as the index is purely "auxiliary", it can be used in tools that take advantage of it without *any* negative backward compatibility consequences for existing tools that are unaware of this index.

© The Author 2024. Published by Oxford University Press. All rights reserved. For permissions, please e-mail: journals.permissions@oup.com

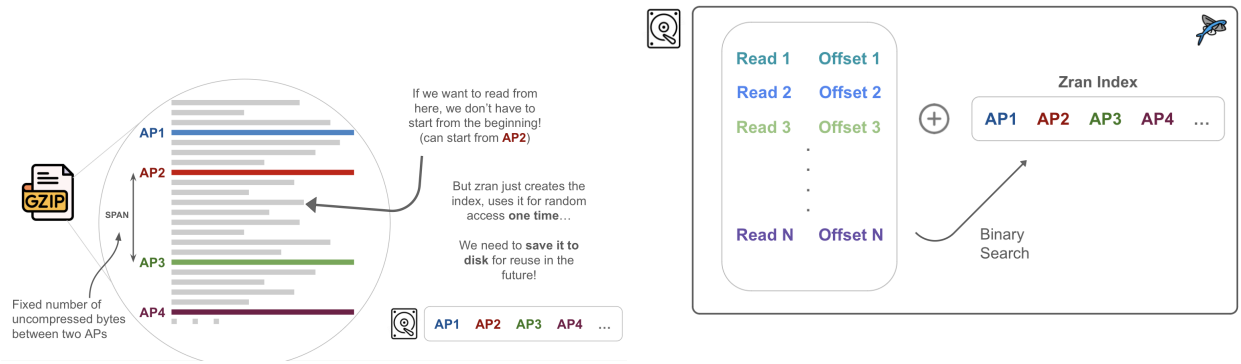

Fig. 1. Structure of Zran Index (left) and FFIndex (right). FFIndex is an FAST-Q file based extension of Zran Index that supports record level access. [The logo for FFIndex is taken from [2]]

The implementation to create the access points and corresponding 32KB uncompressed context is adopted from `zran.c`, which is an example file in the zlib GitHub repository [3]. `zran.c` decompresses the input file in its entirety and creates this index. This index is then retained in memory to allow random access from some byte offset. We store this index data structure to disk in compressed format to allow future processing. This index is also augmented with information on record boundaries to allow random access starting from a record boundary. The access points are saved every few thousand records (tuneable parameter). We show the speedups in processing by using this in a producer-consumer based framework where the producer reads some records from the `FASTQ` file starting at a checkpoint and adds it to the queue. Then the consumer picks up these read records and does the processing over them.

To focus almost entirely on the parsing task, and not introduce non-trivial extra work in actual data processing, the downstream task we use in FFParser for experiments is counting the number of bases in the `FASTQ` file. This application is used because it is not only lightweight but also helps us to sanity check the correctness of our implementation.

TODO:cite RabbitFX

Background

Sequencing data is typically created and shared in the form of `FASTQ` files. To reduce storage and transmission costs, these `FASTQ` files are often compressed using *gzip*, which reduces their sizes. The most prominent example of such a database is the European Nucleotide Archive [16].

These compressed `FASTQ` files are the first step of any pipeline for processing sequencing data. Before being processed, the gzipped `FASTQ` records must be decompressed and parsed, using a library such as (`kseq` [10], or the the parallel, producer-consumer based `FQFeeder`[13]). FQFeeder is a producer-consumer-based implementation, using `kseqpp` [4] for the underlying parser, that uses a producer thread pool for single and paired-end `FASTQ` parsing. The producer threads decompress and parse the records, and places chunks of parsed `FASTQ` records onto a concurrent parsed-record queue. Consumer threads then take these chunks of parsed records and performs any actual computation over the parsed read records (e.g. alignment, quality assessment, etc.).

As seen in Figure 2, the producer thread pool cannot have more threads than the number of files being parsed (since each file is decompressed and parsed sequentially). We see that this is the major scalability bottleneck for the pipeline, which our tool solves. This implementation uses moodycamel's ConcurrentQueue [12] for producer and consumer queues. We want to have a similar producer-consumer structure to our library but where the parsing is replaced with a parallel parser that can make use of an index for `gzip`ped files.

The *gzip* application uses the `DEFLATE` algorithm [15] to compress and decompress binary data. `DEFLATE` consists of two stages. In the first stage, the data is processed sequentially and LZ77 [17] parsing is performed. This encodes the data as a sequence of literals and (off,length) pairs. (off,length) is the offset and length of the longest prefix in preceding 32KB context that can be used to replace data at this location. Once the LZ77 is done, Huffman coding [5] is done to further encode the data compactly. The Huffman code trees are reset every block to give best results. `DEFLATE` decompression is the opposite process - the compressed data is worked upon by Huffman decoding. This writes down the data in a 32KB circular queue, which is then used by LZ77 decoding to decompress the data entirely.

Similar to the standalone `gzip` software application, *zlib* [3] is a software library for compression/decompression of general data streams using the same compression algorithm, DEFLATE. It can be thought of as a generalized version of `gzip`. `zlib` contains an example program in its source tree called `zran.c`. This example program demonstrates the use of certain `zlib` API features by accessing a `gzip`- compressed file from a random offset.

It accomplishes this by fully decompressing the file, and building an index containing "access points" at *uniform* locations. Each "access point" includes the starting file offset, **TODO:**what does this part mean? bit of each deflate block, and 32KB of uncompressed data immediately preceding the point. This index is then used to fetch some number of bytes from an arbitrary, user-provided location in the original `gzip` file (starting at the most-recent access point prior to the desired location). This program does not save or load the index from disk, but rather stores it in memory for a one-time random fetch (`zran` is, after all, an example and not a full application).

	Compressed FASTQ file size	FFIndex file size	FFIndex creation time	FFIndex reading time	% of bytes to store uncompressed states	% of bytes to store offset locations of read records
Salmonella	1.4 MB	39 KB	56 ms	<1ms	99.9%	0.1%
Ecoli	39 MB	5 MB	2819 ms	58 ms	62.9%	37.1%
Dataset C - Illumina	520 MB	38 MB	18846 ms	315 ms	47.5%	52.5%
Nematode	655 MB	48 MB	25816 ms	400 ms	61.2%	38.8%
ENCFF000FEU	8.1GB	543MB	267 s	4.5 s	45%	54%

Table 1. File size and runtime statistics on FFIndex run for various inputs.

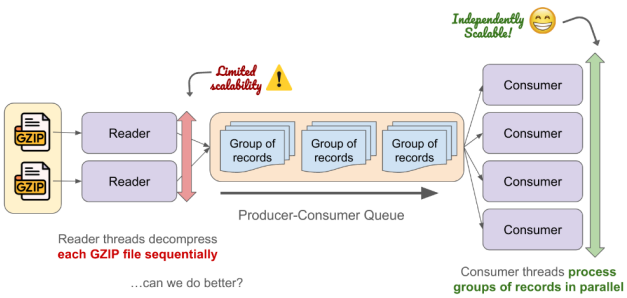

Fig. 2. Structure of FQParser. It can not read a single FASTQ file in parallel.

Related Work

pugz

`pugz` [7] is a parallel algorithm that enables fast decompression of gzipped files. It can achieve this by being able to decompress from a random offset by creating the fully decompressed 32KB context using a two-pass heuristic approach. The 32KB context can be constructed almost always at low compression levels, while some approximations are needed at higher levels.

rapidgzip

TODO:write a paragraph about rapidgzip [8]

mgzip and pgzip

Python libraries built on Python's `zlib` and `gzip` wrappers. `pgzip` [14], a maintained fork of `mgzip` [11], compresses uncompressed buffers by breaking them into blocks, and compressing each block in parallel. The metadata of all blocks is inserted into the `FEXTRA` field within the `gzip` file. This allows for parallel decompression by reading this metadata. While they maintain backward compatibility with `gzip` (as in, the `gzip` utility can decompress the generated `gzip` file), they generate a different `gzip` file due to independent block compression, lacking the desired "original `gzip` + supplementary index file" functionality we want.

Our Approach

FFIndex

In this section we describe our design for FFIndex, the index structure we use to enable parallel parsing of gziped `FASTQ` files.

Base framework

We built our solution on top of the `zran.c` codebase [3], which demonstrates building an index that help access a random offset within a `gzip` file. The main idea behind the index is that DEFLATE algorithm used by `gzip` requires *at most* a 32-KB context window to decompress data at any offset. `Zran` uses this property to build an index that stores access points at approximately fixed spans of the of uncompressed stream. At each access point, `zran` stores up to 32KB of uncompressed data prior to the access point, as well as a small amount of metadata necessary to begin decoding from this position, as well as the current offset into both the compressed and uncompressed streams. For reading from a random offset o in the uncompressed stream, `zran` first performs a binary search on the checkpoints to find the greatest lower bound checkpoint (i.e. the checkpoint with the largest uncompressed offset less than or equal to o). It then begins decompressing the file from this point until it reaches uncompressed byte offset o. Figure 1 (left) shows the structure of the `zran` index.

Augmenting `zran` for sequencing data

`FASTQ` files contains read records that are not of fixed size, and so each record can span a variable number of bytes. we don't handle paired-end reads currently Any application that reads `FASTQ` files needs to read records as atomic blocks and needs to be able to index records to be able to match them in paired-end reads. Therefore, augment `zran`'s index to support record-aligned access. We do so by maintaining, for each `zran` checkpoint, two additional pieces of metadata; the byte offset of the first record occurring at or after this checkpoint (i.e. how far from the checkpoint, in the uncompressed stream, one must read to encounter the start of the first complete record in this chunk) and the record rank corresponding to this record (i.e. starting from 0, how many records have we seen prior to this in the uncompressed stream). To compute this auxiliary information, we also augment the KSeq++ [4] parser to also store, for each parsed record, the byte offset from the start of the file until the record start position. This mapping is built directly after the original `zran` index is built, and is stored alongside the standard `zran` index as a list of access points. Figure 1 (right) shows augmentation of `zran`'s index with our read to offset mapping.

Storing FFIndex

In order for the index to be useful, the compressed file needs to have a sufficient number of checkpoints to saturate many parsing threads. At the same time, we want each chunk not to be *too* large, as each chunk will read and maintain an uncompresesed chunk in memory during parsing. Furthermore, since the index itself has a fairly regular structure, to reduce the

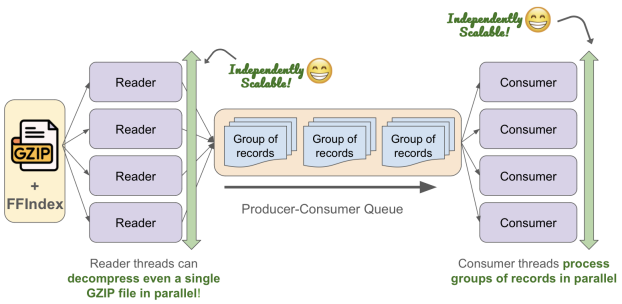

Fig. 3. Structure of FFParser and how it can be used to independently scale on a single FAST-Q file.

size of the index on disk, we compress this as well. Ultimately, we observe the size of the index to be very small; $\sim 0.13\%$ of the size of the compressed data file itself in most cases. In cases where the input file is actually already `bgzip` compressed, we find the index to be even smaller (0.000358% of the size of the file for `pbmc_10k_v3_S1_L001_R2_001`). This is because `zran` chooses its checkpoints naturally to align with the bgzip chunks (each of which is typically much smaller than the chunk sized in our index). Since each `bgzip` chunk requires no additional context to decompress, our index collapses to store basically just the auxiliary chunk to read offset and read rank inforamtion. Index creation itself requires decompressing and parsing the original input file, plus a small amount of bookkeeping to construct the `zran` and auxiliary index itself. Thus, the time to create the `FFIndex` is about twice the time required to simply read the input file. This index creation cost is is a one time cost, therefore we believe that this is a reasonable trade-off. Once created, the indices themselves are very small (the largest we encountered in our testing was $\sim 32MB$, and load within a fraction of a second. Statistics concerning the index sizes and how they relate to the original files are given in**??**.

FFParser

Next, we describe our design for the parallel reader that reads `FASTQ` files taking advantage of the `FFIndex` we have just described. One benefit of the approach described above is that obtaining disjoint and independent, record-aligned chunks of the input file requires almost no synchronization. In our parser, the parser object retains an atomic integer that records the last checkpoint that was provided to a consumer thread. The index is read once from the parser. Likewise, each consumer thread has a read-only copy of the `FFIndex`, as well as a local buffer and its own file handle for the underlying compressed file. When a consumer wishes to obtain a chunk of sequences to read, it requests a chunk from the parser, which hands out the next available chunk and increments the atomic pointer, unless no more unprocessed chunks remain.

The records read are parsed using `KSeq++` [4] and converted to a well formed C++ structure that can be used by downstream tasks. Specifically, we have made a wrapper for `KSeq++`'s input stream type to allow it to treat an uncompressed byte slice (i.e. a chunk decompressed according to the `FFIndex`) directly as an input source. Thus, the process of obtaining a chunk of work for a consumer is trivial. The consumer requests a chunk from the parser. If one is not available, the parser returns false immediately. If one is available, the consumer thread increments the parser's atomic counter and claims the current chunk. To claim the chunk c, it obtains the c-th checkpoint from the `zran` index, which gives the compressed (c_z) and uncompressed (c_u) byte offsets at the start of the c-th chunk, as well as the context necessary to start decompressing the file from offset c_u in the uncompressed stream. At the same time, it obtains the matched record boundary consisting of s_u^c (the byte offset in the uncompressed stream corresponding to the first complete record in the c-th chunk) and r_u^c (the global rank of this record). The consumer thread then begins decompressing the stream from byte c_z (corresponding to c_u in the uncompressed stream), and reads and decompresses until byte $s_u^{(c+1)}$ — the first byte of the first complete record in the *next* chunk, or until the end of the file if c is the last chunk. The consumer then discards the first $s_u^c - c_u$ bytes of what it has decompressed. It then has in its local buffer a slice of $s_u^{(c+1)} - s_u^c$ bytes corresponding to a span of $r_u^{(c+1)} - r_u^c$ records in the uncompressed stream. However, this purely local buffer can then be independently *parsed* by this consumer, while other consumers request remaining chunks of the input file. This is no longer a valid architecture diagram Figure 3 shows a basic structure of FFParser and how it can be used to independently scale on a single FAST-Q file, as opposed to FQFeeder which needs multiple FAST-Q files to scale the number of producers.

Evaluation

In this section, we discuss our experimental setup and the results for benchmarking `FFIndex` and `FFParser`. ADD SPECS for Newton

Experimental setup

We seek to evaluate our approach with respect to two critical criteria: (a) Evaluate how lightweight `FFIndex` is, and (b) Ensure that `FFParser` correct and determine how much parallelism can be achieve achieved.

For the the first aspect, we evaluate the index creation times and reading times, and most importantly the index sizes for a range of `FASTQ` files of different sizes (and structures).

For the evaluation we kept the span of access points to be constant (every $25,000,000$ uncompressed bytes in the underlying stream), which generated different index sizes for different `FASTQ` files. Ideally, we may want to optimize access point placement for a given file size or file structure, but we leave this for future work. We focus on showing that the index size is small enough and with enough access points to allow for parallel reading of the `FASTQ` file with both small and large numbers of threads.

For the second aspect, we evaluate the speedups we can achieve with `FFParser`. We run `FFParser` on 10 different `FASTQ` files, with details given in Table 2. The range of input file sizes ranges from $\sim$ 400MB (compressed) to $\sim$ 24GB (compressed). We run the parser in each case varying the number of threads from 2 through 24.

To keep the focus on the speed of *parsing* and not any computation done with the parsed data, the downstream task used for benchmarking is a simple task of counting the number of occurrences of each nucleotide across all of the records in each of the `FASTQ` files. We compare our tool's performance with a sequential reader using the sem underlying parsing library. We also attempted an experiment using `pgzip` (Python), but it took almost an order of magnitude more time than the `C++`-based

threads dataset	time(s) const	1	2	4	8	12	16	20	24
ENCFF000FFF	255.48	197.51	86.71	55.87	29.85	21.65	16.69	13.18	11.45
ERR1190770_2	105.45	80.94	35.95	21.85	12.89	9.16	7.08	5.65	4.72
SRR13060943_2	231.56	199.61	88.99	65.94	33.73	24.12	19.14	14.44	12.64
SRR26865528_2	651.53	614.67	263.66	230.85	120.92	87.46	64.41	52.13	43.35
SRR28028283_1	93.01	93.32	39.35	36.23	19.51	12.53	9.50	7.28	6.49
SRR28048028	13.26	12.22	5.42	4.04	2.21	1.48	1.22	0.95	0.84
SRR28439552_1	123.88	118.98	52.39	43.58	21.58	15.99	11.18	9.37	7.98
SRR28896749_1	903.93	748.54	315.94	236.24	129.12	79.27	68.21	54.48	46.09
SRR9331208_1	262.56	211.60	92.57	65.26	36.12	25.18	18.15	15.94	12.75
pbmc_10k_v3_S1_L001_R2_001	549.21	472.86	208.24	183.64	105.44	80.04	62.89	50.68	40.70

Table 2. The original (gzipped) file size in bytes, as well as the `ffindex` file size and the percentage of the original file occupied by the corresponding `ffindex` file (at the default threshold of $25,000,000$.

dataset	size (bytes)	index size (bytes)	% of original
ENCFF000FFF	8,841,145,896	11,677,803	0.132%
ERR1190770_2	4,064,792,531	5,357,712	0.132%
SRR13060943_2	8,559,534,660	11,258,189	0.132%
SRR26865528_2	18,395,512,386	24,470,289	0.133%
SRR28028283_1	2,468,520,067	3,309,234	0.134%
SRR28048028	389,868,847	518,384	0.133%
SRR28439552_1	3,622,507,026	4,784,714	0.132%
SRR28896749_1	24,651,189,456	32,655,108	0.132%
SRR9331208_1	7,927,679,468	10,402,795	0.131%
pbmc_10k_v3_S1_L001_R2_001	16,688,669,099	59,694	0.000358%

sequential baseline; therefore we are not using it as a baseline for further evaluation.

Results

Evaluating FFIndex

Table 1 shows the index size for FASTQ files of different sizes. The index size is around 5-10% when compressed, and this size can be adjusted by controlling the number of SPAN bytes after which access points are created. Furthermore, the index size can be reduced by minimizing the data stored for byte offsets of read records. This data constitutes around 40-50% for large FASTQ files. In the current implementation, the byte offset data is stored for each read record. This data is not needed for each record because consecutive records are read by a single reader thread. By storing the offset locations only for the read records of interest, the index size can be significantly reduced. We need to investigate the ideal number of read records after which we can store this data while ensuring strong scaling of performance.

We also report index creation times, which can grow to a few minutes for large files. For example, for 8.1GB ENCFF000FEU FASTQ file, it took 4 minutes to generate the index. However, this is acceptable as it is a one-time cost and can be amortized over multiple runs. The read times are also reported which are significantly less than the overall runtime. This performance can be further improved by reading the index in parallel using the same approach.

Evaluating FFParser

First, we evaluate the correctness of FFParser by comparing the output of our downstream nucleotide counting task with the sequential reader. Figure 5 shows the output of the nucleotide counting task for the ENCFF000FEU file for the sequential reader and FFParser. We can see that the outputs match exactly. We also ran the same test for other FAST-Q files and found the same results. This demonstrates that FFParser is correct and can be used for downstream tasks.

Figure 4 shows the strong scaling results for FFParser on the two different FAST-Q files - Nematode (left) and ENCFF000FEU (right). We can see that FFParser scales near linearly with the number of producer threads for both the files. We also see that FFParser outperforms the sequential reader and FQFeeder with the same number of consumer threads by scaling the number of producer threads.

We notice that for lower producer threads, the performance of FFParser is worse than the sequential reader and FQFeeder. Ideally, our tool should match FQFeeder's performance with the same number of consumer threads and 1 producer thread. We believe that the difference in performance is due to the fact that we have not optimized the producer-consumer queue as well as FQFeeder has. Furthermore, we fix the number of records to be read per producer thread to be 1000. This means that even if we have a single producer thread, we will read 1000 records at a time. Currently, in our implementation, such a case is handled by the thread calling a binary search on the index to find the access point to start from. Clearly, for a single producer thread, this is not needed as we can just read the records sequentially. This case needs to be handled in our implementation by having the ability to start reading directly from the last byte read by that thread if no other thread is reading that block of records. We leave this for future work.

Secondly, we notice that the performance of FFParser varies greatly with the number of access points in the index. This is expected again because every producer thread reads a fixed number of records. If the number of access points $\rightarrow 1$, then

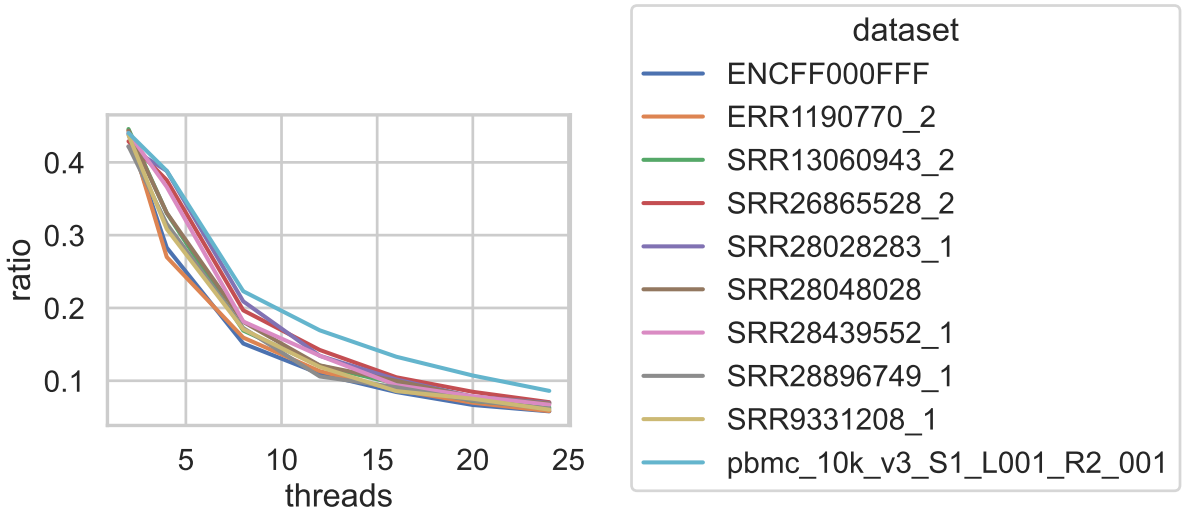

Fig. 4. Strong scaling results for FFParser .

```
→ taskset -c 0-63 ./countbases.out ENCFF000FEU.fastq.gz
Number of records: 125395196
#A: 2349691213
#C: 2218630365
#G: 2696337268
#T: 2263786966
→ taskset -c 0-63 ./test_parser.out ENCFF000FEU.fastq.gz \
ENCFF000FEU.fastq.gz.index.gzip  32 32
Starting parsing
Time to load the gzip index 10197 milliseconds
Parsers Started


#A = 2349691213
#C = 2218630365
#G = 2696337268
#T = 2263786966
```

Fig. 5. Correctness of FFParser. We compare the output of the nucleotide counting task for the sequential reader (top) and FFParser (bottom) on the ENCFF000FEU FAST-Q file. We can see that the outputs match exactly.

it is equivalent to having no index at all and each thread will search for the the nearest access point to start reading from, which in this case is the beginning of the file. Thus each thread will read the file sequentially to get to the block of records it needs to read. This is extremely inefficient. We beleive that this can be alleviated by configuring the number of records read per producer thread as follows:

$$\text{records per thread} \propto \frac{\text{total records}}{\text{number of access points}}$$

This way, each thread will read a larger number of records if the number of access points is less and vice versa. This will also ensure that ideally only one thread reads a block of records between two access points. We were not able to implement this in our current version of FFParser and leave this for future work.

Despite, these identified bottlenecks in the implementation (which are easy to fix), we can see that FFParser scales extremely well and still outperforms the sequential reader and FQFeeder by almost 10x with the largest count of producer threads in each experiment. This demonstrates that FFParser greatly benefits from the parallelism introduced by the FFIndex.

Limitations and Future Work

To further demonstrate the effectiveness of FFIndex and FFParser, as well as to study scalability bottlenecks, we can test our implementations using FAST-Q files of other sizes. Because building the FFIndex is a one time work, efforts should be spent optimizing the runtime of FFParser instead. One way of accomplishing this is by collecting detailed traces of FFParser's executions, and understanding where the bottlenecks are located.

The most significant potential for future optimizations is reducing the size of FFIndex. Since FFIndex builds on zran's index construction algorithm, we currently create access points that are around SPAN bytes apart. We also store a mapping of record boundaries to their respective byte offsets in the gzipped file. One potential optimization that may be possible is to construct access points exactly at record boundaries, so that these two data structures can be represented as one, helping us reduce the size of the index.

Each of the reader threads of FFParser performs a binary search on the access points to determine where to start decompressing the gzipped file. By collapsing the two data structures, we could theoretically avoid this binary search. However, these binary searches are performed independently in parallel, and take a very insignificant amount of time compared to downstream tasks that dominate in runtime. Because this increase in complexity may not be worth the runtime benefit, we are not sure this is worth considering in further work.

Lastly, the producer-consumer model in FFParser is a proof-of-concept of how FFIndex can be utilized in an end-to-end Fast-Q processing pipeline. However, our evaluation from section 5 shows that the current implementation of FFParser is not optimised for cases where the number of producer threads is very small and the number of access points are small. We have identified the cause for this and present easy potential solutions to these problems in section 5. We were not able to incorporate these solutions in our current implementation due to time constraints. The implementation of our architecture is therefore, not production ready and is to show the potential of FFIndex in a parallel processing pipeline. We also beleive a more productive use of time would be to adding Index as a plug-in to FQFeeder, which does have an optimized producer-consumer pipeline. Using FFParser, FQFeeder can fully exploit parallelism and avoid the scalability bottleneck in reading files, if provided the FFIndex.

Conclusion

Our work addresses a significant bottleneck faced by members of the genomics community: efficiently parsing large, compressed FASTQ files in parallel. In this project, we present FFIndex, a lightweight supplementary index to gzipped FASTQ files that enables the parsing of a single highly compressed FASTQ file in parallel.

To demonstrate its utility, we also implemented FFParser, a multi-threaded FASTQ parser that achieves a near-linear speedup compared to serial parsing. By using FFIndex, each of the readers in FFParser can parse groups of records within a compressed FASTQ file in parallel, and enqueue them for downstream processing. Our results show that FFParser achieves strong scaling as we add the number of producers.

By overcoming the bottleneck of needing to parse each FASTQ file sequentially, our work enables much better utilization of available modern hardware. We hope that researchers find FFIndex and FFParser useful in speeding up their workflows, cutting costs, and potentially making prohibitive tasks accessible.

References

1. P. Deutsch. *DEFLATE Compressed Data Format Specification version 1.3*. May 1996.
2. Flaticon. Flaticon. `https://www.flaticon.com/free-icon/flying-fish_6255787?term=flying+fish&page=1&position=3&origin=search&related_id=6255787`, 2024.
3. Jean-loup Gailly and Mark Adler. zlib. `https://www.zlib.net/`, 2024.
4. Ali Ghaffaari. KSeq++. `https://github.com/cartoonist/kseqpp`.
5. David Huffman. A Method for the Construction of Minimum-Redundancy Codes. *Proceedings of the IRE*, 40(9):1098–1101, September 1952.
6. Maël Kerbiriou and Rayan Chikhi. Parallel decompression of gzip-compressed files and random access to DNA sequences. In *2019 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 209–217. IEEE, 2019.
7. Maël Kerbiriou. pugz. `https://github.com/Piezoid/pugz`.
8. Maximilian Knespel and Holger Brunst. Rapidgzip: Parallel Decompression and Seeking in Gzip Files Using Cache Prefetching. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing*, HPDC '23, page 295–307. ACM, August 2023.
9. Ben Langmead, Christopher Wilks, Valentin Antonescu, and Rone Charles. Scaling read aligners to hundreds of threads on general-purpose processors. *Bioinformatics*, 35(3):421–432, 2019.
10. Heng Li. seqtk. `https://github.com/lh3/seqtk`, 2013.
11. Vincent Li. mgzip. `https://github.com/vinlyx/mgzip`.
12. moodycamel. moodycamel's concurrentqueue. `https://github.com/cameron314/concurrentqueue`.
13. Rob Patro. FQFeeder. `https://github.com/rob-p/FQFeeder`.
14. pgzip. pgzip. `https://github.com/pgzip/pgzip`.
15. KATZ PHILLIP W. String searcher, and compressor using same.
16. David Yuan, Alisha Ahamed, Josephine Burgin, Carla Cummins, Rajkumar Devraj, Khadim Gueye, Dipayan Gupta, Vikas Gupta, Muhammad Haseeb, Maira Ihsan, Eugene Ivanov, Suran Jayathilaka, Vishnukumar Balavenkataraman Kadhirvelu, Manish Kumar, Ankur Lathi, Rasko Leinonen, Jasmine McKinnon, Lili Meszaros, Colman O'Cathail, Dennis Ouma, Joana Paupério, Stephane Pesant, Nadim Rahman, Gabriele Rinck, Sandeep Selvakumar, Swati Suman, Yanisa Sunthornyotin, Marianna Ventouratou, Senthilnathan Vijayaraja, Zahra Waheed, Peter Woollard, Ahmad Zyoud, Tony Burdett, and Guy Cochrane. The European Nucleotide Archive in 2023. *Nucleic Acids Research*, 52(D1):D92–D97, November 2023.
17. J. Ziv and A. Lempel. A universal algorithm for sequential data compression. *IEEE Transactions on Information Theory*, 23(3):337–343, May 1977.